Lecture 2c

From Words to Vision — Embeddings and CLIP

Feature	DALL·E 1	DALL·E 2	DALL·E 3	Stable Diffusion
Year	2021	2022	2023	2022
Architecture	Transformer + VQVAE	CLIP + Diffusion + Decoder	Transformer + Diffusion (end-to-end)	CLIP + Latent Diffusion (U-Net)
Latent Space?	Yes (VQ-VAE)	Yes (CLIP + diffusion latent)	Yes (diffusion latent)	Yes (VAE latent space)
Text Guidance	Built-in	CLIP (separate)	Transformer prompt guidance	CLIP text encoder
Open Source?	❌	❌	❌	✅ (by CompVis)
Resolution	256×256	1024×1024 (with upsampling)	Up to 1024×1024	512×512, 768×768 (v2+), SDXL

Why Multimodal Embeddings?

🎯 The Challenge

- **Text** and **images** live in fundamentally different spaces:
  - Text: sequences of tokens, syntactic structure
  - Images: spatial pixel arrays, visual semantics

> 🔁 How can we make models understand *that*
> "a red apple" ↔ 🍎 and "a cute puppy" ↔ 🐶?

Why Multimodal Embeddings?

🎯 The Challenge

- **Text** and **images** live in fundamentally different spaces:
  - Text: sequences of tokens, syntactic structure
  - Images: spatial pixel arrays, visual semantics

> 🔁 How can we make models understand *that* "a red apple" ↔ 🍎 and "a cute puppy" ↔ 🐶?

🧠 Goal:

Learn a **shared representation space** where:

- Text and image pairs that describe the same concept are **close together**
- Unrelated text and images are **far apart**

Why Multimodal Embeddings?

🎯 The Challenge

- **Text** and **images** live in fundamentally different spaces:
  - Text: sequences of tokens, syntactic structure
  - Images: spatial pixel arrays, visual semantics

> 🔁 How can we make models understand *that* "a red apple" ↔ 🍎 and "a cute puppy" ↔ 🐶?

🧠 Goal:

Learn a **shared representation space** where:

- Text and image pairs that describe the same concept are **close together**
- Unrelated text and images are **far apart**

📌 Benefits of a Shared Embedding Space

- Enables **semantic similarity** across modalities
  (e.g. text ↔ image, image ↔ text, or image ↔ image)
- Supports **zero-shot classification** and **retrieval**
- Forms the foundation for **text-conditioned generation** in DALL·E 2

Why Multimodal Embeddings?

🎯 The Challenge

- **Text** and **images** live in fundamentally different spaces:
  - Text: sequences of tokens, syntactic structure
  - Images: spatial pixel arrays, visual semantics

> 🔁 How can we make models understand *that* "a red apple" ↔ 🍎 and "a cute puppy" ↔ 🐶?

🧠 Goal:

Learn a **shared representation space** where:

- Text and image pairs that describe the same concept are **close together**
- Unrelated text and images are **far apart**

📌 Benefits of a Shared Embedding Space

- Enables **semantic similarity** across modalities
  (e.g. text ↔ image, image ↔ text, or image ↔ image)
- Supports **zero-shot classification** and **retrieval**

ned generation in DALL·E 2

💬 Analogy:

> Like translating English and French into a common meaning space 🌍 — "dog" and "chien" land near each other

CLIP – Contrastive Language–Image Pretraining

📅 Historical Context

- 📍 **Released by OpenAI in January 2021**, around the same time as DALL·E 1
  (Paper: "Learning Transferable Visual Models From Natural Language Supervision")
- 🔄 Trained on **400 million (image, text)** pairs from the internet
- Aimed to **learn from natural supervision**: captions, alt-text, memes, etc.
- CLIP was a **breakthrough** in learning **general-purpose, zero-shot visual models** using **language as supervision** — no manual class labels needed!

CLIP – Contrastive Language–Image Pretraining

📅 Historical Context

- 📍 **Released by OpenAI in January 2021**, around the same time as DALL·E 1
  (Paper: "Learning Transferable Visual Models From Natural Language Supervision")
- 🔄 Trained on **400 million (image, text)** pairs from the internet
- Aimed to **learn from natural supervision**: captions, alt-text, memes, etc.
- CLIP was a **breakthrough** in learning **general-purpose, zero-shot visual models** using **language as supervision** — no manual class labels needed!

🧠 Core Idea of CLIP

> Learn to embed **text** and **images** into the **same semantic space**,
> so that matching pairs land close together, and mismatches stay far apart.

CLIP – Contrastive Language–Image Pretraining

📅 Historical Context

- 📍 **Released by OpenAI in January 2021**, around the same time as DALL·E 1
  (Paper: "Learning Transferable Visual Models From Natural Language Supervision")
- 🔄 Trained on **400 million (image, text)** pairs from the internet
- Aimed to **learn from natural supervision**: captions, alt-text, memes, etc.
- CLIP was a **breakthrough** in learning **general-purpose, zero-shot visual models** using **language as supervision** — no manual class labels needed!

🧠 Core Idea of CLIP

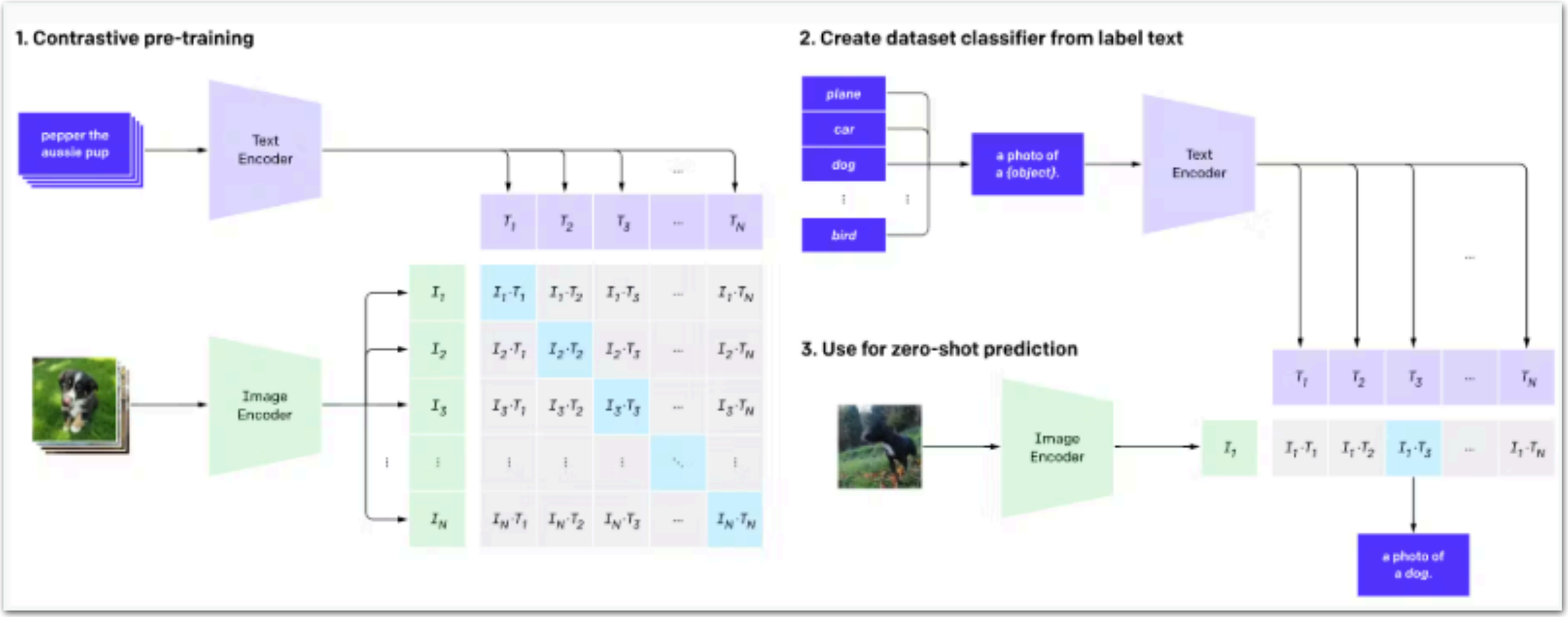
> Learn to embed **text** and **images** into the **same semantic space**,
> so that matching pairs land close together, and mismatches stay far apart.

🏗️ Architecture: Dual Encoders

Modality	Encoder	Output
Text	Transformer (BPE)	$\mathbf{t} \in \mathbb{R}^d$
Image	ResNet / ViT	$\mathbf{i} \in \mathbb{R}^d$

- Both produce a **single vector** per input (text or image)
- Trained to **maximize similarity** between matching text–image pairs

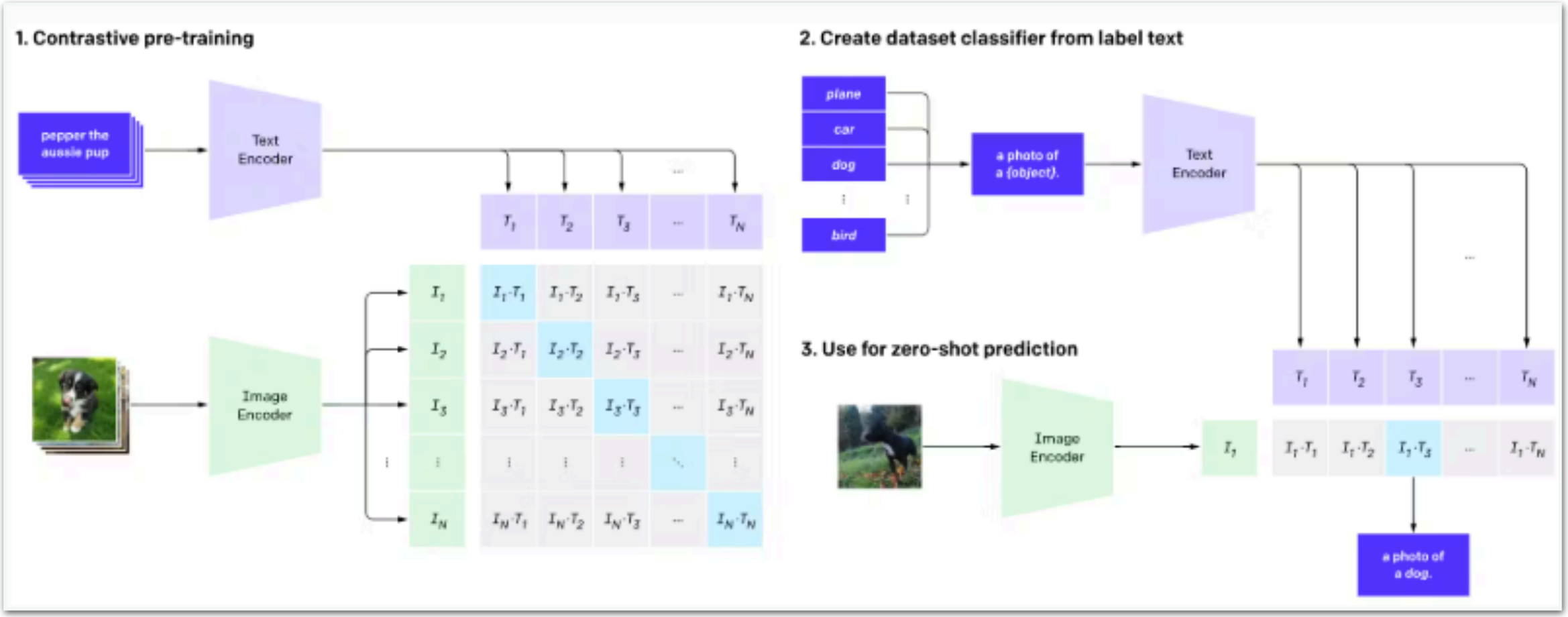
CLIP



Key Components:

- **Image Encoder:**
  - Options: **ResNet-50** or **Vision Transformer (ViT-B/32)**.
  - Outputs: 512-dimensional **image embedding** vector.
- **Text Encoder:**
  - A **Transformer** similar to GPT or BERT.
  - Encodes the **entire caption** into a single 512-dim **text embedding**.

CLIP

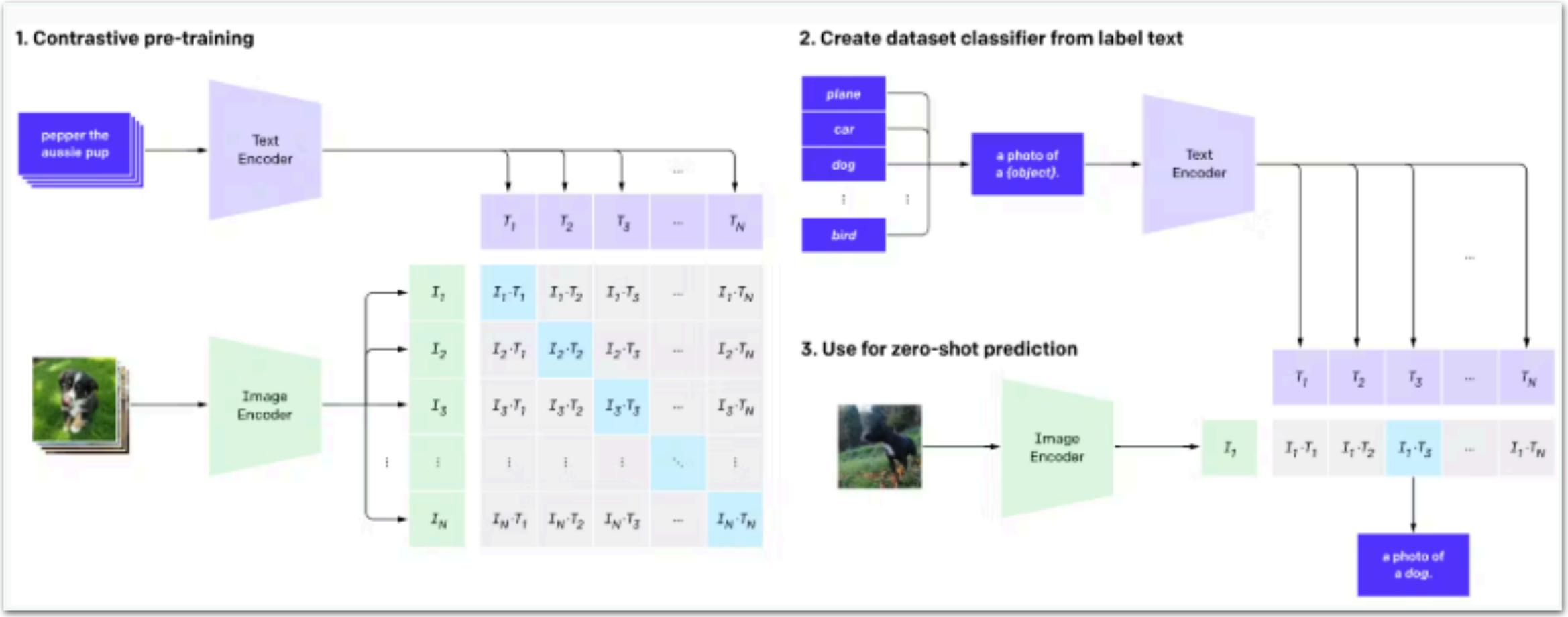


- **Shared Embedding Space:**
  - Both modalities are projected into the **same vector space**.
  - Allows **semantic similarity** comparison between image and text.

Contrastive Training Objective:

- During training, CLIP maximizes the similarity of **correct (image, text)** pairs and minimizes similarity for **incorrect** ones.
- Implemented via a **contrastive loss** over all pairs in a batch.

CLIP

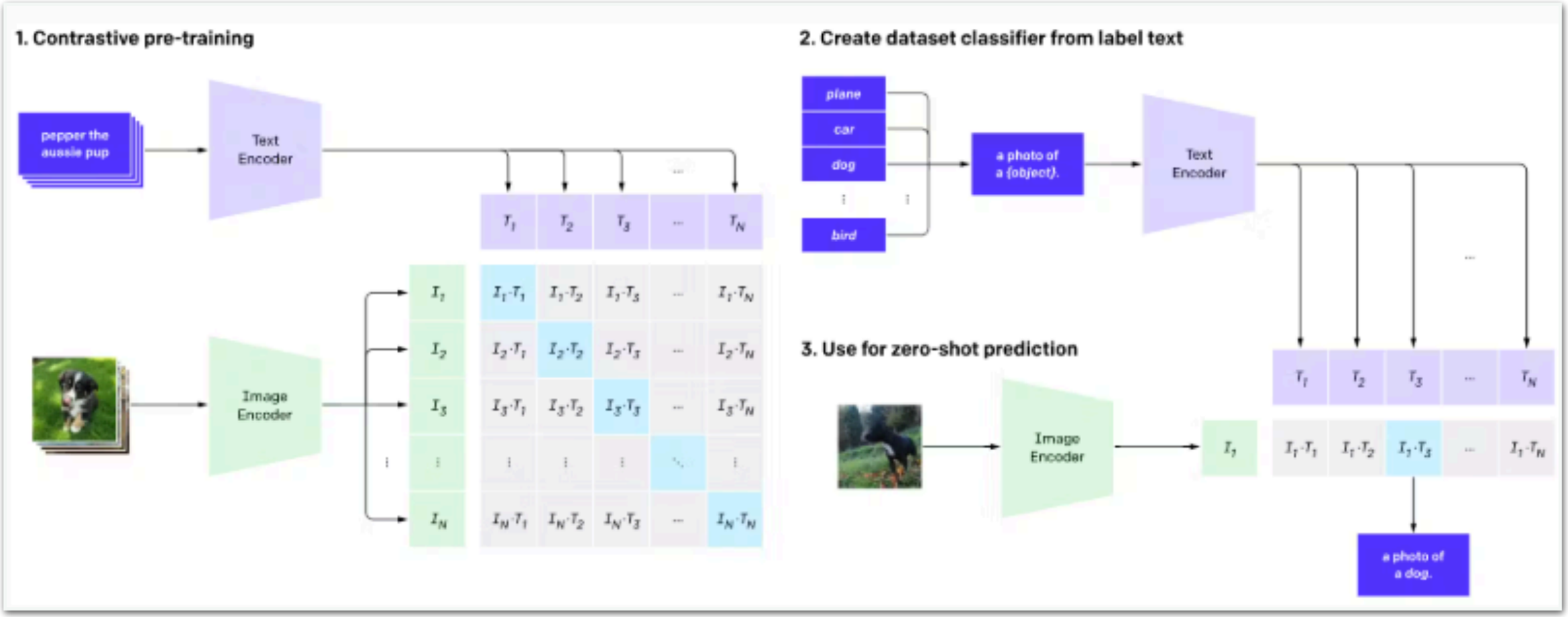


- **Shared Embedding Space:**
  - Both modalities are projected into the **same vector space**.
  - Allows **semantic similarity** comparison between image and text.

Contrastive Training Objective:

- During training, CLIP maximizes the similarity of **correct (image, text)** pairs and minimizes similarity for **incorrect** ones.
- Implemented via a **contrastive loss** over all pairs in a batch.

CLIP

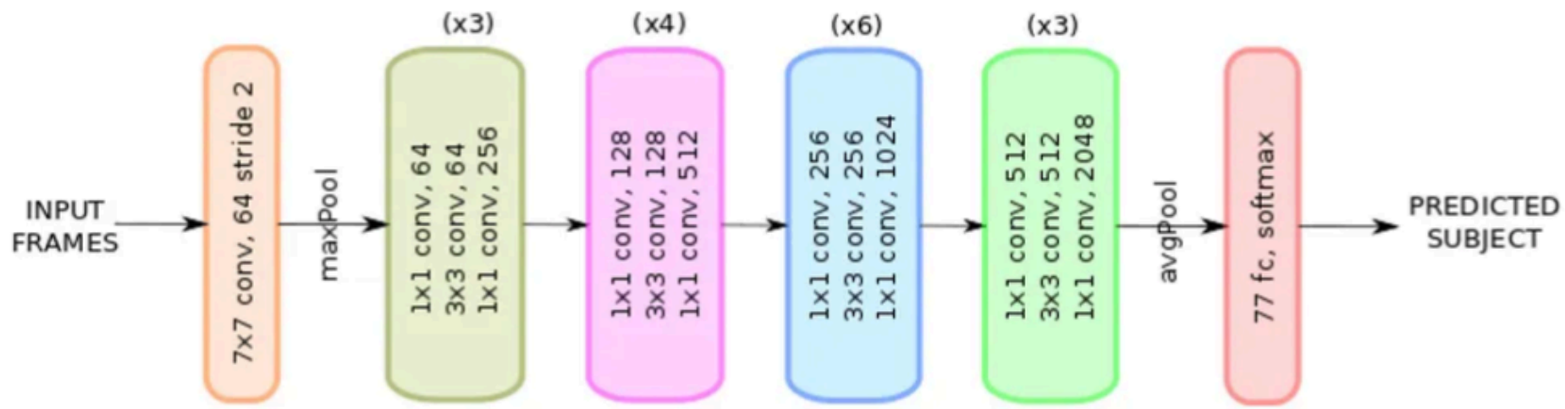


Key Components:

- **Image Encoder:**
  - Options: **ResNet-50** or **Vision Transformer (ViT-B/32)**.
  - Outputs: 512-dimensional **image embedding** vector.
- **Text Encoder:**
  - A **Transformer** similar to GPT or BERT.
  - Encodes the **entire caption** into a single 512-dim **text embedding**.

Now more on these...

ResNet-50: Deep CNN with Skip Connections



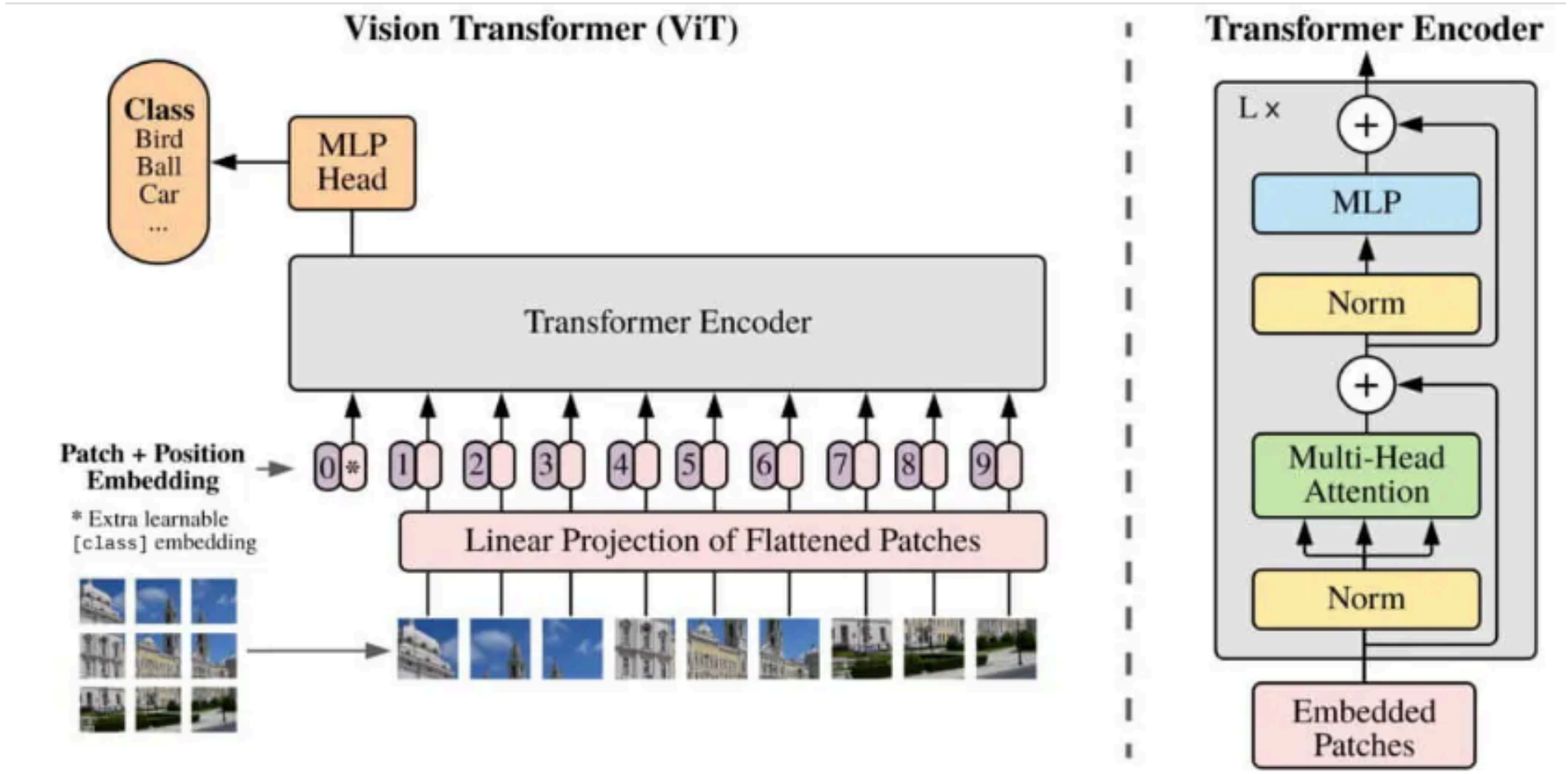
1 + 9 + 12 + 18 + 9 + 1 = 50 layers

Key Ideas:

- Introduced in **2015** by He et al., ResNet won the ImageNet challenge.
- ResNet-50 is a **50-layer convolutional neural network**.
- It uses **residual connections**: instead of learning full mappings, the model learns **residuals** (differences).
- This allows **very deep networks** to be trained without vanishing gradients.

Output:

- Final output is a **512-dimensional embedding vector**.

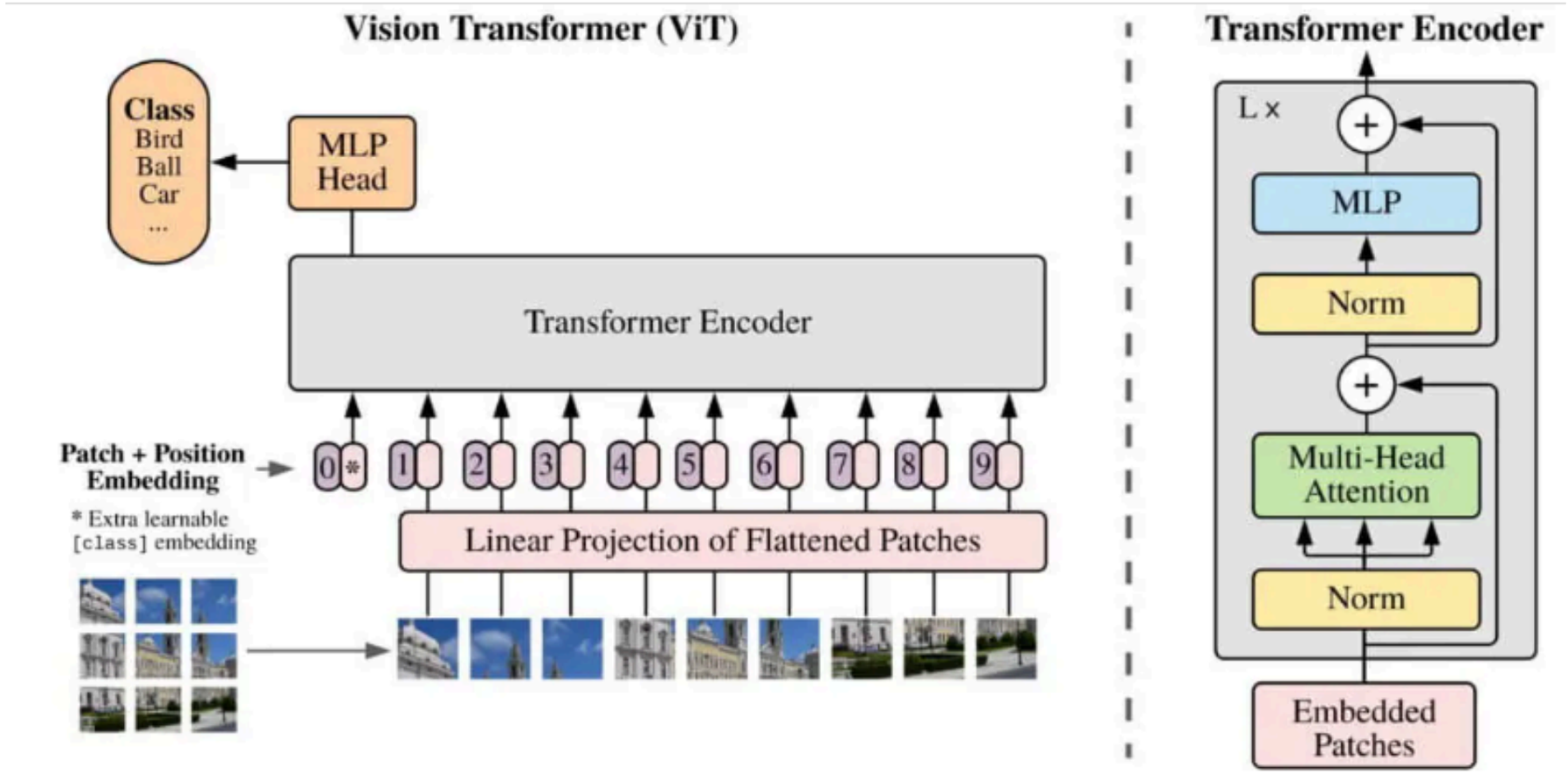


Key Ideas:

- Introduced by **Dosovitskiy et al., 2020**.
- Treats an image as a sequence of **patches** — like tokens in NLP.

ViT Workflow:

1. Image is divided into **fixed-size patches** (e.g. 16×16).
2. Each patch is **flattened and linearly projected** into an embedding.
3. Position embeddings are added.
4. The patch sequence is passed through a **Transformer encoder**.
5. The output of the **[CLS] token** is used for classification or embedding.

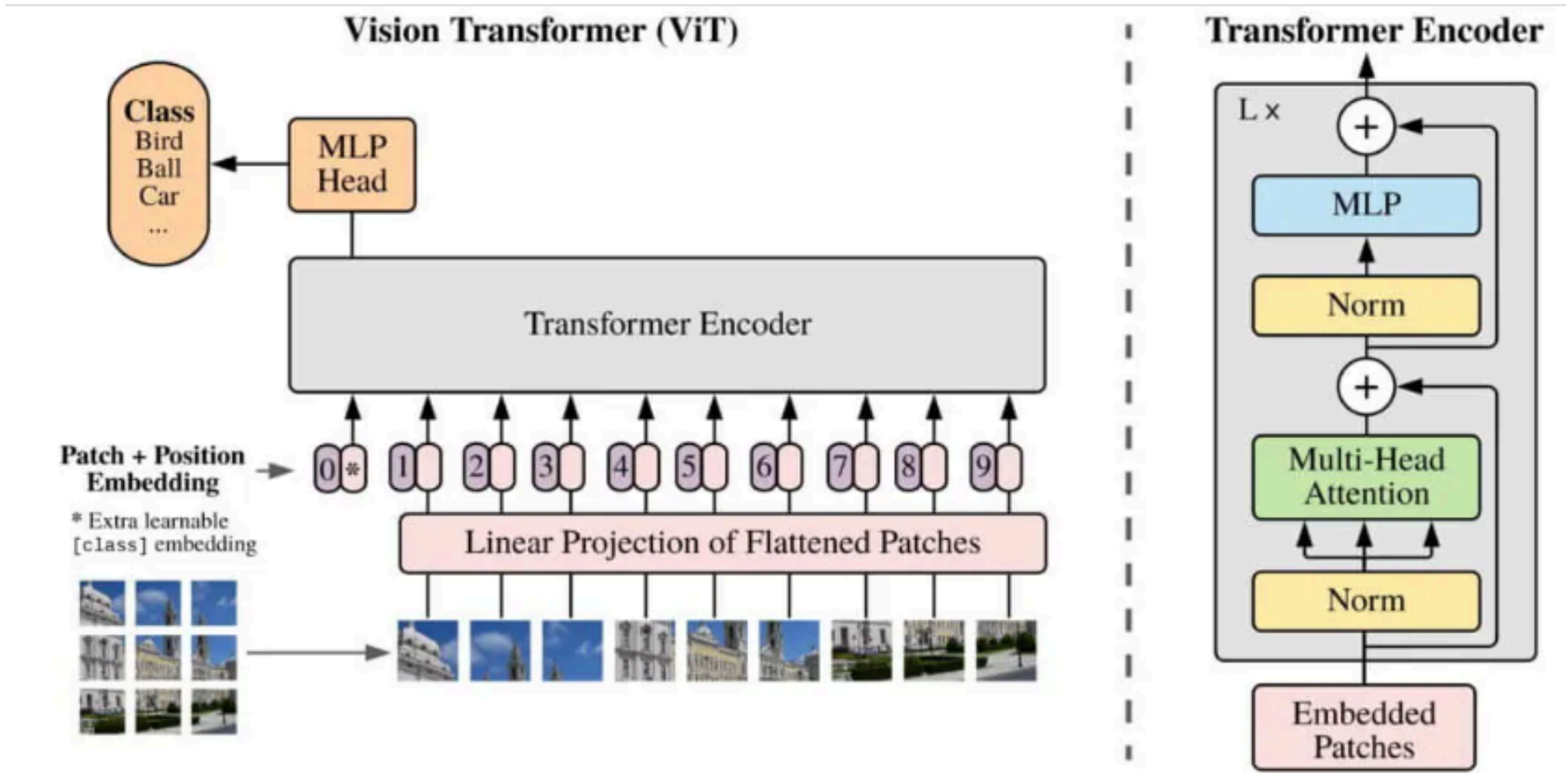


CLIP uses ViT-B/32:

- Patch size = 32×32 (coarser than standard ViT).
- Faster and more scalable than ResNet at large scale.
- Output: **512-dimensional embedding**.

Why use ViT in CLIP?

- Unified architecture for **both image and text** (both use Transformers).
- Captures **global dependencies** well.



Now we can continue with the loss.....

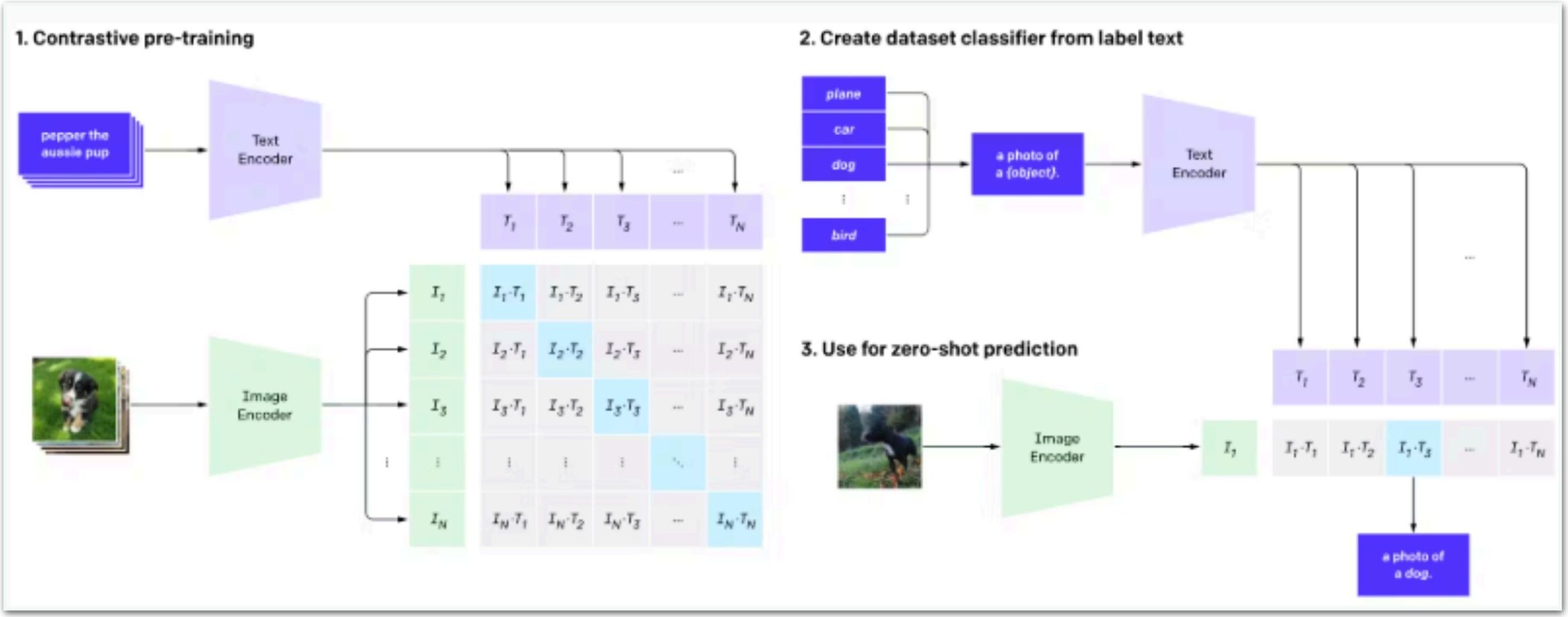
CLIP uses ViT-B/32:

- Patch size = 32×32 (coarser than standard ViT).
- Faster and more scalable than ResNet at large scale.
- Output: **512-dimensional embedding**.

Why use ViT in CLIP?

- Unified architecture for **both image and text** (both use Transformers).
- Captures **global dependencies** well.

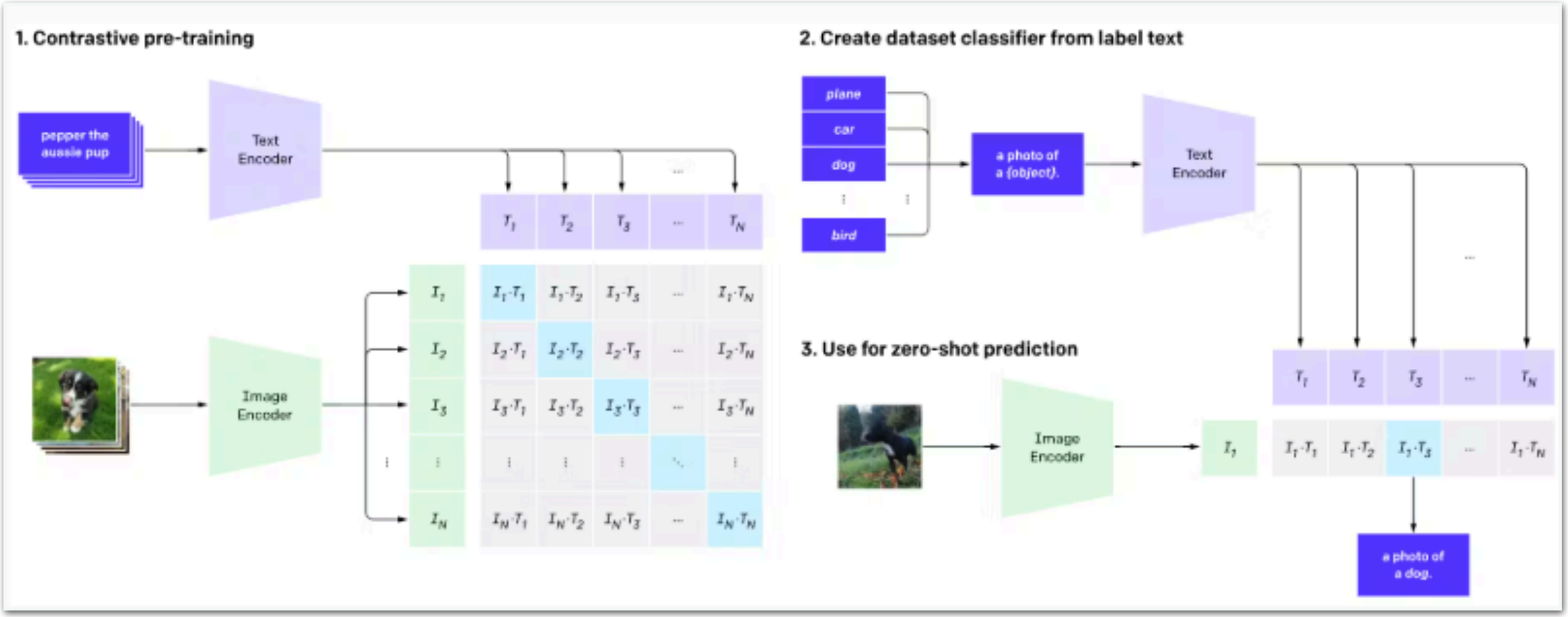
CLIP



Key Components:

- **Image Encoder:**
  - Options: **ResNet-50** or **Vision Transformer (ViT-B/32)**. Just saw...
  - Outputs: 512-dimensional **image embedding** vector.
- **Text Encoder:**
  - A **Transformer** similar to GPT or BERT. Skip now...just a very good text encoder.....
  - Encodes the **entire caption** into a single 512-dim **text embedding**.

CLIP



- **Shared Embedding Space:**
  - Both modalities are projected into the **same vector space**.
  - Allows **semantic similarity** comparison between image and text.

Contrastive Training Objective:

More on this....

- During training, CLIP maximizes the similarity of **correct (image, text)** pairs and minimizes similarity for **incorrect** ones.
- Implemented via a **contrastive loss** over all pairs in a batch.

CLIP's Contrastive Loss — Matching Images and Text

Goal:

- For a batch of size **N**, CLIP learns to **align each image with its matching text**, and vice versa.
- It uses a **contrastive loss** that rewards correct pairs and penalizes mismatched ones.

CLIP's Contrastive Loss — Matching Images and Text

Goal:

- For a batch of size **N**, CLIP learns to **align each image with its matching text**, and vice versa.
- It uses a **contrastive loss** that rewards correct pairs and penalizes mismatched ones.

Key Equations:

Given:

- I_i: image embedding for sample i
- T_i: corresponding text embedding
- Similarity: $s_{ij} = \langle I_i, T_j \rangle$ (dot product)

Image-to-Text loss:

$$\mathcal{L}_{\text{img}}^{(i)} = -\log \frac{\exp(s_{ii}/\tau)}{\sum_{j=1}^{N} \exp(s_{ij}/\tau)}$$

Text-to-Image loss:

$$\mathcal{L}_{\text{text}}^{(i)} = -\log \frac{\exp(s_{ii}/\tau)}{\sum_{j=1}^{N} \exp(s_{ji}/\tau)}$$

- τ: temperature hyperparameter (learned or fixed)

Final Loss:

$$\mathcal{L} = \frac{1}{2N} \sum_{i=1}^{N} \left(\mathcal{L}_{\text{img}}^{(i)} + \mathcal{L}_{\text{text}}^{(i)} \right)$$

CLIP Capabilities — From Training to Downstream Tasks

1. Zero-Shot Classification

How it works:

- At test time, you provide **class names as text prompts**:
  - e.g., "a photo of a cat", "a photo of a dog"
- CLIP embeds these texts and compares them to the **image embedding**.
- The class with **highest similarity** is chosen — **no fine-tuning needed**.

CLIP Capabilities — From Training to Downstream Tasks

2. Image-Text Retrieval

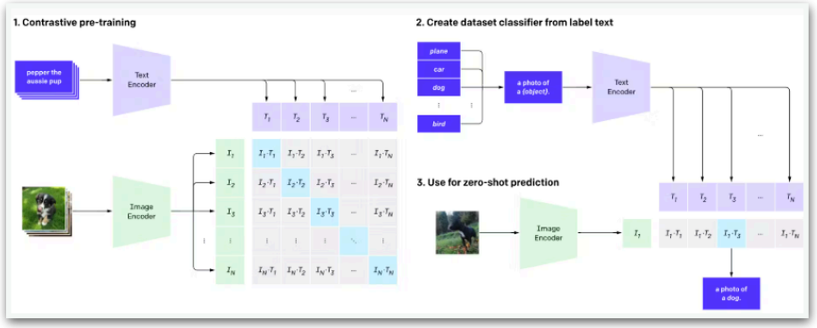
- **Text → Image**: Find the most relevant image to a text prompt.
- **Image → Text**: Find the best caption for a given image.
- Done via **nearest neighbors in embedding space**.

CLIP Capabilities — From Training to Downstream Tasks

3. Multimodal Tasks Built on CLIP

- Text-to-image generation (**DALL·E 2, Stable Diffusion**).
- Prompt engineering and fine-tuning (e.g., **CLIP-guided** models).
- Image editing via text (e.g., "change sky to sunset").

Understanding the Shared Embedding Space



What Happens After Training?

- Both images and texts are projected into a **shared 512-dimensional vector space**.
- Similar **meanings** (not appearances!) cluster together.
- The space is **semantic** — e.g., photos, sketches, or drawings of a "cat" will be close to the caption "a photo of a cat".

CLIP: A Foundation for Multimodal Intelligence

Key Takeaways:

- **CLIP = Contrastive Language–Image Pretraining**
  - Trained to **match images and their textual descriptions**.
  - Uses **contrastive loss** over large batches (InfoNCE).
- **Dual Encoder Architecture:**
  - **Image Encoder:** ResNet or ViT
  - **Text Encoder:** Transformer
  - Both output vectors in a **shared embedding space**.
- **No labels required** — trained on 400M internet (image, caption) pairs.
- **Zero-shot learning:**
  - Just change the **text prompt** — no retraining needed.
- **Enables powerful downstream tasks:**
  - Classification, retrieval, image generation, prompt design.
- **Foundation for DALL·E 2, Stable Diffusion, Midjourney**, and more.